

AWS NAT Gateway & Microservices Architecture

Summary: NAT Gateway, ALB, and Order-to-Payment Flow

What is a NAT Gateway?

A NAT (Network Address Translation) gateway lets devices/servers in a private subnet reach the internet for outbound requests, without exposing them to direct inbound traffic. It translates a private IP to a public IP for outgoing requests and routes the response back to the originating device. Traffic only flows out and back — unsolicited inbound connections are blocked.

NAT Gateway vs. ALB

These solve different problems and are unrelated — creating one does not create the other.

- **ALB (Application Load Balancer):** handles inbound traffic, distributing requests from the internet to backend targets (EC2, ECS, Lambda).
- **NAT Gateway:** handles outbound traffic, letting private-subnet resources initiate connections to the internet.

An ALB usually sits in a public subnet to be reachable via an Internet Gateway. A NAT gateway is used separately for resources needing outbound-only internet access.

NAT Use Case: Spring Boot Microservice

A typical microservice (e.g., order-service) runs in a private subnet for security. Inbound requests arrive via the ALB — no NAT involved. But the service often needs outbound access: calling third-party APIs (Stripe, Twilio, SendGrid), pulling dependencies from Maven Central, or reaching AWS services without a VPC endpoint. Since the private subnet has no public IP, this outbound traffic routes through the NAT gateway, which translates the request and routes the response back. Note: internal service-to-service and service-to-database calls within the VPC never touch NAT.

Cost note: NAT gateways charge per hour plus per GB processed. For AWS-only destinations (e.g., S3, DynamoDB), a VPC endpoint is often cheaper than routing through NAT.

Order-to-Payment Flow

1. Client places an order → request hits the ALB (public subnet, only internet-facing entry point).
2. ALB routes to order-service (private subnet).
3. order-service validates the order (checks inventory internally, saves to DB) — all internal VPC traffic, no NAT.
4. order-service calls payment-service, directly or via an event queue (SQS/SNS/Kafka).
5. payment-service calls an external payment gateway (e.g., Stripe) — this outbound call routes through the NAT gateway.
6. Payment gateway responds (see below for how the response returns) and may later send an async webhook.
7. payment-service updates status and notifies order-service (internal call or event).
8. order-service finalizes the order; notification-service may email/SMS the customer (also via NAT if using an external provider).
9. Response flows back to the client, synchronously or via async notification.

How Stripe's Response Reaches order-service

Path 1 — Synchronous API response:

payment-service opens an outbound HTTPS connection to Stripe through the NAT gateway. From Stripe's perspective, the request source is the NAT gateway's public IP — Stripe is not aware of the ALB or NAT gateway at all. Stripe simply sends its HTTP response back on the same open connection. The NAT gateway is stateful: it matches the returning traffic against its connection-tracking table and routes it back to payment-service's private IP. This is return traffic for a connection payment-service itself initiated, not new inbound traffic, so it's allowed through automatically. The ALB plays no role here.

Path 2 — Asynchronous webhook:

For delayed confirmations (e.g., bank transfers, disputes), Stripe later initiates a brand-new connection into your system. A NAT gateway cannot accept this since it only permits return traffic for connections initiated from inside the VPC. Instead, the webhook must hit a public endpoint: Stripe → ALB (internet-facing) → a webhook-handler target in a private subnet, which verifies Stripe's signature and updates payment/order status. Either path ends the same way: the final status is communicated to order-service via an internal call or event, moving the order to confirmed or failed.